I2I Agentic AI Starter Guide

Executive Study Summary

A concise reference distilled from the I2I Agentic AI Starter Guide (FDE). Intended for quick revision and shared learning across the team.

---

1. Purpose and Prerequisites

The starter guide provides a shared language for designing and building agentic systems: single agents, multi-agent collaboration, workflows, tools, memory, guardrails, and protocols (MCP and A2A). Before deep implementation work, readers should understand three foundations:

- Large Language Models (LLMs) as reasoning and decision engines inside agents.
- APIs and external tools - how agents reach live systems of record.
- Programming fundamentals (typically Python) for hands-on agent loops and integrations.

2. Core Distinctions (Agentic Nuances)

AI Agent: A single LLM-driven system that can decide and act, usually via tools.
Agentic AI: A capability style/spectrum: planning, dynamic tool or agent choice, memory, adaptation, and often collaboration - not merely a fixed script.
Multi-agent workflow: Multiple agents with roles (orchestrator + specialists, pipeline, hierarchical). More agents are not automatically more intelligent.
Important nuance: a fixed A->B->C multi-agent pipeline without autonomous decision-making is workflow automation, not true agentic intelligence. Use a single agent for linear tasks (FAQ, one-step API). Use agentic designs for multi-step, open-ended, cross-domain work. Avoid over-engineering simple problems.

Spectrum: Scripted bot -> tool-using agent -> agentic planner -> multi-agent system.

3. Orchestration Styles

- Workflow-driven: predefined graph/state machine - predictable, auditable, easier to test.
- Autonomous / LLM-orchestrated: the model chooses next tools or agents at runtime - flexible, harder to guarantee.
- Hybrid (usual enterprise choice): deterministic rails for identity, policy, and writes; LLM freedom for drafting, ranking, and research.

An orchestrator decomposes goals, delegates to tools or specialists, merges results, and decides when to stop or escalate (including human-in-the-loop).

4. Application Building Blocks

Typical stack: Channel/UI -> Orchestration -> Agent(s) <-> Memory -> Tools/MCP/other agents (A2A) -> Systems of record. Cross-cutting concerns: guardrails, observability, evaluations, and identity.

- LLM roles: understand intent, plan, select tools, fill arguments, synthesize answers, stop/continue/ask.
- Tools: name, description, parameter schema, implementation, permissions. Narrow reliable tools beat one mega-tool.
- ReAct-style loop: Reason -> Act (tool) -> Observe -> repeat until final answer or escalation.
- Critical security rule: authorization and irreversible actions must be enforced in code/tool layers, not only in the system prompt.

5. Memory and Guardrails

LLMs are effectively stateless across calls. Memory types include short-term/working context, episodic session history, semantic long-term facts, procedural playbooks, and shared team memory. Start with session memory; add long-term only when clearly valuable - and protect privacy/retention.

Guardrails constrain unsafe or invalid behavior: input filters, output filters, tool allowlists and AuthZ, business policy

checks, scope limits, and cost/loop limits (max tool calls, timeouts). HITL (human-in-the-loop) is intentional for high-impact actions - not a failure of autonomy. Prefer risk-tiered autonomy: freer on read/draft; stricter on write/send/pay.

6. MCP (Model Context Protocol)

MCP standardizes how AI hosts/agents connect to tools and context sources (files, APIs, databases, SaaS) without inventing a custom integration format every time. Analogy: USB for capabilities.

- Roles: Host (app), Client (speaks MCP), Server (exposes tools/resources/prompts).
- Local function tools are fine for small prototypes; MCP value grows with reuse across teams and hosts.
- Security: treat MCP servers like APIs - authentication, least privilege, server-side validation, audit logs.

7. A2A (Agent-to-Agent) and Combined Use

A2A addresses discovery, messaging, and collaboration between agents (capabilities/cards, tasks, results, progress updates, auth). Use multi-agent/A2A when specialist skills, system ownership, or organizational boundaries require it - not because it sounds advanced.

Complementary split: A2A coordinates agents; MCP (or equivalent tools) supplies each agent with systems access.

- Failure modes: agent ping-pong, ambiguous contracts, auth gaps, latency/cost explosion.
- Mitigations: max hops, clear ownership, structured schemas, timeouts, budgets.

8. Platforms and Feature Tiers

Compare platforms by workflow control, memory depth, tool/MCP support, observability/evals, deployment/SSO fit, and team skill (code-first vs low-code) - not by brand alone.

Must-have before production

- Tool calling with schemas; AuthN/AuthZ; logging/tracing; guardrails on high-risk actions; timeouts/error handling.

Should-have

- Appropriate memory; HITL gates; evaluation harness; cost/latency controls; versioning of prompts/graphs.

Could-have / scale

- Multi-agent/A2A, MCP tool mesh, advanced adaptive planning, richer analytics.

Recommended build order: one use case and metric -> tools + auth -> simple loop/workflow -> guardrails/HITL -> observability + golden evals -> memory if needed -> multi-agent/MCP/A2A when forced by integration needs.

9. Production Stack Essentials

- AI gateway: centralized model access, keys, rate limits, fallbacks.
- Orchestration engine: graphs, state, retries, human gates.
- Retrieval (RAG/search): grounds answers in approved content - RAG alone is not a full agent.
- Observability: per-run traces of prompts, tools, latency, errors.
- Evals: regression checks on task success, tool choice, safety - distinct from live tracing.
- Identity and secrets: agents act as someone; make that explicit and auditable.

10. Key Takeaways for Design Reviews

- Say how much autonomy you have - do not stop at 'we use Agentic AI'.
- Prefer hybrid rails for compliance-critical steps.
- Tool/schema quality and AuthZ often matter more than model upgrades.
- MCP != A2A; more agents != better.
- Ship tools, guardrails, observability, and evals before exotic multi-agent designs.

11. Quick Reference Glossary

ReAct: Reason -> Act (tool) -> Observe -> repeat.

HITL: Human-in-the-loop approval for high-impact actions.
Guardrail: Hard check that keeps behavior safe and valid (input, output, tool, policy, cost).
ACI: Agent-Computer Interface - treat tool design like clear API design for the model.
Hallucination: Model invents facts; mitigate with tools for live data, retrieval for approved docs, schemas, and evals.

12. One-Paragraph Design Pitch

We build hybrid agentic systems: deterministic workflows for compliance-critical steps, LLM agents for flexible reasoning, narrow authenticated tools (optionally via MCP), guardrails and HITL on writes, and traces plus golden evals so changes do not silently regress. Multi-agent or A2A is introduced only when specialist ownership or trust boundaries truly require it.

Source: I2I Agentic AI Starter Guide (FDE). This summary is a learning aid for internal study and discussion; refer to the original guide for full detail and platform tables.